

A Testing Scheme for Self-Adaptive Software Systems with Architectural Runtime Models

Joachim Hänsel, Thomas Vogel and Holger Giese
Hasso Plattner Institute for Software Systems Engineering
at the University of Potsdam, Potsdam, Germany

Email: [Joachim.Haensel|Thomas.Vogel|Holger.Giese]@hpi.de

Abstract—Self-adaptive software systems (SASS) are equipped with feedback loops to adapt autonomously to changes of the software or environment. In established fields, such as embedded software, sophisticated approaches have been developed to systematically study feedback loops early during the development. In order to cover the particularities of feedback, techniques like one-way and in-the-loop simulation and testing have been included. However, a related approach to systematically test SASS is currently lacking. In this paper we therefore propose a systematic testing scheme for SASS that allows engineers to test the feedback loops early in the development by exploiting architectural runtime models. These models that are available early in the development are commonly used by the activities of a feedback loop at runtime and they provide a suitable high-level abstraction to describe test inputs as well as expected test results. We further outline our ideas with some initial evaluation results by means of a small case study.

I. INTRODUCTION

Traditionally, software development follows an open-loop structure that requires human supervision when software systems are exposed to changing environments [1]. To reduce human supervision, software systems are equipped with feedback loops to adapt autonomously to changing environments. Such closed-loop systems are designated as self-adaptive software systems (SASS) [2] and they are often split in two parts, an *adaptation engine* realizing the feedback loops and controlling the *adaptable software* [1]. As pointed out by Calinescu [3], such systems will become important for safety-critical applications, where they have to fulfill high-quality standards.

Testing is an established technique for ensuring quality in traditional systems, even safety-critical ones [4], and processes for testing such systems exist. For instance, embedded software with its feedback loops is often systematically tested in three stages [5, pp. 193–208]: i) a simulation stage that tests the models (specification) of the software under development in a simulated or real-life environment, ii) a prototyping stage that tests the real software in a simulated environment, and finally, iii) pre-production stage that tests the real software in the real environment. With each stage the software is more and more refined to the final product while testing continuously provides assurances for the software and particularly early in the development.

However, a similar systematic testing process providing continuous and early assurances does not exist for SASS. In contrast, models for substituting the environment or parts of a SASS usually cannot be obtained easily and therefore,

a generic simulation environment for SASS does not exist. Consequently, testing SASS typically requires that the implementations of the feedback loops and adaptable software with its sensors and effectors are available. This impedes testing early in the development and makes it costly to remove faults in the feedback loops discovered late in the development.

Furthermore, approaches used for traditional systems are not as easily applicable to SASS as the interface between the adaptation engine and the adaptable software is often quite different from that of embedded software. SASS are usually not restricted to observing and adjusting parameters but additionally monitor and adapt the architecture of the software [6]–[8], thus requiring support of structural adaptations [9].

Some approaches address the testing of SASS but only for later development stages [10]–[14] when the systems have already been deployed. Others do promote testing in earlier stages but they still assume an executable and complete SASS to run the tests against [15]–[17]. Testing of only parts of the feedback loop is not supported. In contrast, we consider testing parts of a SASS as a precondition to early validation since a system with the completely implemented adaptable software and feedback loop is only available in the latest development stages.

Therefore, we propose a systematic testing scheme for SASS that allows engineers to test the feedback loops (adaptation behavior) early in the development by exploiting runtime models. Such models represent the adaptable software and environment and they are typically used at runtime to drive the adaptation [18]. Our approach leverages early testing of SASS by using *architectural* runtime models that are available early in the development and are commonly used by the activities of a feedback loop. Therefore, feedback loop activities such as monitor, analyze, plan, and execute (cf. MAPE-K [19]) can be individually tested while the whole feedback loop and the adaptable software do not have to be implemented yet. In contrast, the non-implemented parts are simulated based on the runtime models. Consequently, the feedback loop can be modularly tested while the different parts of the loop can be incrementally refined and implemented until they replace the simulated parts. Moreover, we expect reduced costs of testing since we do not require final or experimental implementations of certain feedback loops parts to test other parts.

The rest of the paper is structured as follows. We describe preliminaries in Section II and the benefits of runtime models

for testing in Section III. Then, we discuss our approach by means of one-way (Section IV), in-the-loop (Section V), and online (Section VI) testing. Finally, we sketch an initial evaluation in Section VII, contrast our approach with related work in Section VIII, and conclude the paper in Section IX.

II. PRELIMINARIES

In this section we discuss preliminaries of the presented testing: MAPE-K feedback loops and architectural RTM.

A. MAPE-K Feedback Loops

The development of SASS typically follows the *external approach* [1] that separates adaptation from domain concerns by splitting up the software in two parts: an *Adaptation Engine* for the adaptation concerns and an *Adaptable Software* for the domain concerns while the former *senses* and *effects* and thus, controls the latter. This constitutes a *feedback loop* that realizes the self-adaptation (see Figure 1). The engine senses as well the *Environment* with which the adaptable software *interacts*.

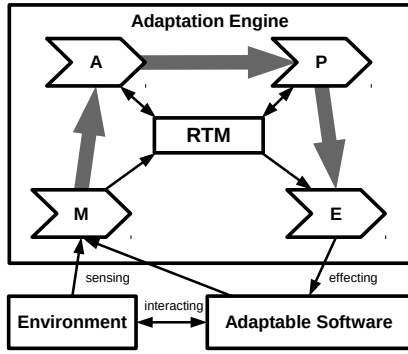


Figure 1. MAPE-K Feedback Loop with a Runtime Model (RTM).

The resulting feedback loop between the engine and the software can be refined according to the *MAPE-K* reference model [19]. This model considers the activities of **M**onitoring and **A**nalyzing the software and environment and, if needed, of **P**lanning and **E**xecuting adaptations to the software. All activities share a **K**nowledge base as illustrated by a runtime model (*RTM*) in Figure 1 and discussed in the following.

B. Architectural Runtime Models

The external approach as previously discussed requires that the adaptation engine has a representation of the adaptable software and environment to perform self-adaptation. This representation is often realized by a causally connected **R**untime **M**odel (*RTM*) [18]. A causal connection means that changes of the software or environment are reflected in the model and changes of the model are reflected in the software (but, not in the environment being a non-controllable entity).

Considering Figure 1, an RTM can be used as a knowledge base on which the MAPE activities are operating. The monitor step observes the software and environment and updates the RTM accordingly. The analyze step then reasons on the RTM to identify any need for adaptation. Such a need is addressed

by the plan step to prescribe an adaptation in the RTM, which is eventually enacted to the software by the execute step.

Using RTMs in self-adaptive software provides the benefits of creating appropriate abstractions of runtime phenomena that are manageable by the feedback loops and of applying automated model-driven engineering (MDE) techniques [18].

The software architecture has been identified as such an appropriate abstraction level for representing the adaptable software and environment and for supporting structural adaptation [6]–[9,18]. Hence, *architectural* RTMs of the adaptable software are used by a feedback loop to reflect on the state of the software and environment. Such state-aware models can be enriched by a feedback loop to cover, for instance, the history or time series of states and executed adaptations, which results in history-/time-aware models.

In our research on self-adaptive software such as [20], we evaluate our work by using *mRUBiS*¹, an internet marketplace on which users sell or auction products, as the adaptable software. A single shop on the marketplace consists of 18 components and we may scale up the number of shops. For a self-healing scenario, we created architectural runtime models of *mRUBiS* and defined different types of failures based on the models. These failures have to be handled by the adaptation engine. Examples of such failures are exceptions emitted by components, unwanted life-cycle changes of components, the complete removal of components because of crashes, and repeated occurrences of these failures. Based on that, we experiment with different adaptation mechanisms and can also exploit the models for testing as discussed in the following.

III. EXPLOITING RUNTIME MODELS FOR TESTING

In the following, we assume a SASS that follows the MAPE-K cycle with runtime models (RTMs) as schematically depicted in Figure 1. If the RTMs are just self-aware and reflect the current state of the adaptable software and environment, we can make the following two observations:

(1) The behavior of the system can be described by a sequence of steps $(\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^* \rightarrow_M \rightarrow_A \rightarrow_P \rightarrow_E (\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^*; \dots$ where $\rightarrow_{AS}$ denotes a step of the adaptable software, $\rightarrow_{ENV}$ denotes a step of the environment, $\rightarrow_M$ denotes the complete monitoring step, $\rightarrow_A$ denotes the complete analysis step, $\rightarrow_P$ denotes the complete planning step, and $\rightarrow_E$ denotes the complete execute step.

(2) The interface between those steps can be described by different states S_i of the RTM if we do not consider the input of the monitoring and the output of the execute step: $(\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^* \rightarrow_M S_1^M \rightarrow_A S_1^A \rightarrow_P S_1^P \rightarrow_E (\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^*; \rightarrow_M S_2^M \dots$ where S_i^M denotes the RTM state after the i -th monitoring, S_i^A the RTM state after the i -th analysis, and S_i^P the RTM state after the i -th planning.

Consider the self-healing example in Figure 2. An intact architecture is monitored and results in RTM S_{i-1}^M . For now, analysis and planning are not required to take action since the architecture is not broken. Without an adaptation, the execute

¹Modular Rice University Bidding System: <http://www.mdlab.de>

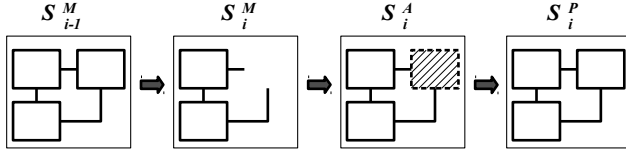


Figure 2. Example Trace for a Self-Healing Scenario.

step will do nothing either. We can directly proceed with the next steps in the environment or adaptable software. Due to either an environmental influence or some failure in the adaptable software ($\rightarrow_{ENV}$ or $\rightarrow_{AS}$), a component of the architecture is removed. In the next step, this is monitored as RTM S_i^M . The result of the analysis step $\rightarrow_A$ is the annotated RTM S_i^A that marks the missing component. The planning step $\rightarrow_P$ constructs a repaired RTM S_i^P which will be applied to the adaptable software in the next step by $\rightarrow_E$.

These two observations indicate that the different states of the RTM are the key element to describe the input/output behavior of the MAPE activities concerning their communication with the adaptable software. Moreover, the RTMs also facilitate considering the required behavior of the adaptation engine at a much higher level of abstraction than the events observed by the monitoring step and the effects triggered by the execute step.² Consequently, we suggest exploiting the RTMs to systematically test the adaptation engine and its parts in form of one-way testing of individual steps and fragments, in-the-loop testing of the analysis and planning steps, and online testing of the analysis and planning steps. We further study how we can validate the model which is required for the in-the-loop testing.

IV. ONE-WAY TESTING

We define *One-Way Testing* as the following: An input RTM and an expected oracle RTM are provided. One or more steps are tested in a single execution of a partial feedback loop. The tested parts receive the input RTM and are supposed to produce an output RTM. The output RTM is compared against the oracle. In this kind of testing the steps $\rightarrow_{AS}$, $\rightarrow_{ENV}$, $\rightarrow_M$, $\rightarrow_A$, $\rightarrow_P$ or $\rightarrow_E$ will happen at most once.

A. One-Way Testing single MAPE Activities

The most basic approach is to test each of the steps/activities that process the RTM on their own. Obviously, these tests need to be run before testing combinations of feedback-loop steps to better locate faults and tell single-step errors from errors that arise due to problems in the interaction of steps.

1) *One-Way Testing the Analysis*: If we want to test the analysis step, we simply provide an input RTM S_1^M , run step $\rightarrow_A$, and compare the resulting RTM S_1^A with an oracle RTM S_o^A . Applied to the example in Figure 2, we choose S_i^M with

²We ignore here the case that the adaptable software and environment change while the feedback loop is running. While this case could not be excluded in general, we may neglect it due to the considered abstraction level as supported by architectural runtime models. That is, oftentimes the architecture does not change very frequently, for instance, due to failures.

the removed component as an input RTM. We then define an oracle RTM S_o^A that contains an annotation where the missing component has been marked. Applying $\rightarrow_A$ on S_i^M would give us S_i^A which is compared to S_o^A . If both RTMs are the same, that is, both especially contain the same “missing component” annotation, the test would pass, otherwise fail.

2) *One-Way Testing the Planning*: Similar to the analysis step, we provide an input RTM S_1^A , run step $\rightarrow_P$, and check whether the output of $\rightarrow_P$ is equal to an oracle RTM S_o^P that was defined before. In the example of Figure 2, we start out with the annotated RTM S_i^A . The oracle S_o^P would be defined as the intact architecture from the beginning (S_{i-1}^M) and we would expect $\rightarrow_P$ to return an RTM equal to S_o^P , that is, the plan step has re-created the removed component in the RTM.

B. One-Way Testing MAPE Fragments

We now discuss one-way testing of fragments by jointly testing the analyze and plan or the monitor and execute steps.

1) *One-Way Testing the Analysis and Planning*: As a precondition to the separate test of the analysis and planning, it is necessary to have knowledge about the way the analysis works and what kind of models to expect. Obviously it would be hard to create a valid oracle model S_o^A or input model S_1^M if this knowledge is not available. In a simple scenario like the self-healing one presented before this should not pose a problem. But there are also more complex analysis algorithms, which will not result in models that can be tested as easily. Furthermore, some errors might only appear if the analysis and planning are tested together.

Consequently, we propose to test the analyze and plan steps as the next unit. Again we can benefit from the same pattern of testing, that is, by providing an input model S_1^M and an oracle model in state S_o^P . In terms of the example trace (Figure 2), this means to start with the broken monitored input model S_i^M , construct an expected model S_o^P where the removed component is redeployed and check whether the resulting model of the application of $S_i^M \rightarrow_{A \rightarrow P} S_i^P$ is equal to S_o^P .

2) *One-Way Testing the Execute and Monitor*: The separate testing of the monitor and execute steps via the runtime models is not feasible as the effect of the execute step cannot be directly observed. If we follow the same pattern as with the analysis and planning, we would end up with no result model for the execute step and no input model for the monitor step. The effect of the execute step cannot be directly observed since it is part of the concrete adaptable software. Likewise, the monitor step’s input is directly obtained from the software. Instead of the separate testing, we propose to test the monitor and execute steps together. In this setup we need a working adaptable software and the tested execute and monitor steps are effecting and sensing the software. The test input is provided by a model S_1^P to the execute step $\rightarrow_E$ which will effect the adaptable software. The adaptable software is monitored $\rightarrow_M$ and a new runtime model is obtained S_2^M .

Equality and inequality of these two models can be interpreted in different ways: (1) equal models may indicate that the monitor and execute steps work correctly, (2) equal models

may also mean that a failure in the execute step is masked by a failure in the monitor step (or the other way round), or (3) that the adaptable software or the environment mask a fault of the execute and/or monitor steps. If S_1^P and S_2^M are not equal, then either (4) the execute step, (5) the monitor step or (6) both do not work properly or (7) the environment introduced an error or the adaptable software showed erroneous behavior.

Cases (3) and (7) can be ruled out by applying the test several times. It is unlikely that the environment will introduce the same error for all test runs and if the adaptable software was tested before, it is equally unlikely that it will constantly show erroneous behavior. In the cases (4), (5) and (6) we can assume a broken monitor and/or execute step. Case (1) should be more likely than (2) since it is not impossible but hard to have two faults that mask each other. Case (2) should become less likely the more tests with different S_1^P and S_2^M are done. In the end, equal models are a good indicator of working execute and monitor steps and non-equal models show that at least one of them is broken.

With this test setup, only parts of the monitoring capabilities can be tested since its purpose is to detect not only correct but also incorrect states of the adaptable software. On the other hand, the execute step is not intended to have an effect on the software that causes an incorrect state. Therefore, we need to be able to impose an “incorrect” RTM on the adaptable software (such as S_i^M in Figure 2), so that we can test whether the monitor step is able to properly observe this incorrect state and create the according RTM. A special test adapter is needed, so that first a correct RTM can be imposed by the execute step and then the incorrect parts are added by the test adapter. The incorrect input RTM S_{1err} needs to be split into S_{1valid} which will be provided to $\rightarrow_E$ and $S_{1invalid}$ which is given to the test adapter. The oracle S_o^M for this test looks like S_{1err} and the monitor should observe an incorrect RTM.

V. IN-THE-LOOP TESTING

Considering the analysis and planning, one-way testing is effective to find errors that always show up, independent from their previous executions in the feedback loop. If we want to identify errors that arise from an accumulated state of the system, we need to test them with sequences of inputs. It would be a cumbersome task to construct these sequences by hand. Instead we propose to provide a simulation that captures the behavior of the adaptable software (AS), environment (ENV), monitor (M) and execute (E) steps. This simulation will provide sequences of RTMs to the analyze step and will read back the RTMs from the plan step. We define such a **runtime model simulation** by an automaton $RTMS = (\mathcal{S}_{RTMS}, \rightarrow_{RTMS})$ that comprises the combined behavior of AS, ENV, M, and E. Note that $RTMS$ is a simulation for testing purposes. The provided input RTM and the way the simulation model reacts to the output of $\rightarrow_A$ and $\rightarrow_P$ are supposed to be realistic but not an exact replacement for the real AS, ENV, M and E. It also means that it may behave non-deterministically to reflect realistic AS and ENV and therefore involves some random component.

In order to decide whether a test is successful, we also need an oracle. In the simplest case the oracle is given by a state property ϕ for the model. In more complex cases ϕ may be even a sequence property or ensemble property. With respect to our example, the oracle may be the sequence property that some architectural constraints for our RTM are only violated for at most n subsequent states.

A. Black-Box In-the-Loop Testing of Analysis and Planning

With $RTMS$ at hand we can test the feedback loop already in an early stage when neither the adaptable software or the monitor and execute steps are available or ready. The analyze and plan steps combined with $RTMS$ can be simulated together and produce observable sequences: $\rightarrow_{RTMS} S_1 \rightarrow_A S_1^A \rightarrow_P S_1^P \rightarrow_{RTMS} S_2 \dots$. From these we consider only the traces of states: $\pi = S_1; S_1^A; S_2 \dots$ and check whether $\pi \models \phi$ to ensure that $\rightarrow_A$ and $\rightarrow_P$ as a black box work as expected.

B. Grey-Box In-the-Loop Testing of Analysis and Planning

We can also aim for a better fault location if we consider the result of $\rightarrow_A$ (i.e., the analyze and plan steps as a grey box). The sequence, we would like to look at, is the following: $\rightarrow_{RTMS} S_1 \rightarrow_A S_1^A \rightarrow_P S_1^P \rightarrow_{RTMS} S_2 \dots$. Here we will inspect the trace $\pi' = S_1; S_1^A; S_1^P; S_2 \dots$. In order to test these traces, we need a property ϕ' that covers S_i^A as well. We now require $\pi' \models \phi'$ to ensure that $\rightarrow_A$ and $\rightarrow_P$ work as expected.

VI. ONLINE TESTING AND VALIDATION

In a later development stage we can reuse the simulation model $RTMS$ and the properties ϕ and ϕ' alongside the running system for online testing and validation.

A. Online Testing

If $\rightarrow_A$ and $\rightarrow_P$ in the running system will expose S_i^A and S_i^P in the same way as in the development stage, we can check ϕ and ϕ' online or against a recorded trace. The simulation is simply replaced with the real system. Whether online or offline testing is to be preferred will depend on available resources on the system under test and the existence of logging facilities. Both approaches, black-box and grey-box testing, are applicable and can be carried out in the same way as with the simulation.

The sequences will be $(\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^* \rightarrow_M S_1^M \rightarrow_A S_1^A \rightarrow_P S_1^P \rightarrow_E (\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^*; \rightarrow_M S_2^M \dots$ and the traces will be the exchanged RTMs: $\pi = S_1^M; S_1^A; S_1^P; S_2^M \dots$

B. Validation

The in-the-loop testing heavily depends on $RTMS$. If an error is detected during in-the-loop testing, it is likely that it is caused by an erroneous adaptation ($\rightarrow_A$, $\rightarrow_P$ or both). But the $RTMS$ itself might also be the source of an error or might mask an erroneous adaptation. The validation of $RTMS$ in this later stage can give an indication about the quality of $RTMS$ and therefore the suitability for testing. Additionally, if the real system produces sequences not covered by $RTMS$ which cause errors in the adaptation, we exactly know which

sequence reveals the error and it can be added to *RTMS* for regression tests.

The idea behind validating *RTMS* is to observe $(\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^* \rightarrow_M S_1^M \rightarrow_A S_1^A \rightarrow_P S_1^P \rightarrow_E (\rightarrow_{AS} \text{ or } \rightarrow_{ENV})^* \rightarrow_M S_2^M \dots$ and look at the traces $\pi' = S_1^M; S_1^P; S_2^M \dots$. If our simulation model *RTMS* is correct, it should cover the observed behavior: $\pi' \in \mathcal{L}(RTMS)$.

VII. INITIAL EVALUATION

In this section, we report on our initial evaluation of the testing scheme for SASS we are proposing in this paper. This evaluation shows the benefits of using (architectural) runtime models with respect to implementing a test framework by means of reusing MDE techniques. Moreover, it gives us preliminary confidence about the effectivity of the scheme when developing feedback loops.

A. One-Way Testing

To realize one-way testing, we developed a generic test adapter that loads the input model, triggers the adaptation steps such as analysis and planning to be tested, and finally, compares the resulting model with the oracle model. Developing such a test adapter has been simplified due to MDE principles as realized by the *Eclipse Modeling Framework* (EMF)³. EMF provides mechanisms to generically load and process models and particularly of comparing models⁴. Hence, we easily obtain matches and differences between two models such as the output model of adaptation steps and the oracle model to obtain the testing result. This result, that is, the output of the comparison, is also a model that can be further analyzed. For instance, the *Object Constraint Language* (OCL)⁵ can be used to check application-specific constraints such as mission-critical components like for authenticating users on the mRUBiS marketplace are not missing in the architecture.

B. In-the-Loop Testing

For the internet marketplace mRUBiS we developed a simulator based on an architectural runtime model. It simulates the marketplace itself (i.e., the adaptable software) thereby injecting failures as well as the monitor and execute steps. The simulator maintains the runtime model against which the analyze and plan steps are developed.

Using this simulator, we can test the analyze and plan steps as follows: i) the simulator injects failures into the runtime model (this simulates the behavior of the adaptable software and environment as well as the monitor step that reflects the failure in the model). ii) the analyze and plan steps to be tested are executed and they analyze and adjust the model according to the adaptation need. iii) the simulator performs the execute step that emulates the effects of the adaptation as performed by the analyze and plan steps in the runtime model. For instance, response times are updated in the model if the configuration of the architecture is adapted.

³EMF: <https://eclipse.org/modeling/emf/>

⁴EMF Compare: <https://www.eclipse.org/emf/compare/>

⁵Eclipse OCL: <http://projects.eclipse.org/projects/modeling.mdt.ocel>

After one run of the feedback loop and before injecting the next failures, the simulator checks whether the analyze and plan steps performed a well-defined adaptation (e.g., by checking whether the life cycle of components has not been violated when adding or removing components) and it checks whether the state of the runtime model represents a valid architecture (e.g., components are not missing or there are no unsatisfied required interfaces, that is, no dangling edges). These checks are performed based on constraints and properties that the runtime models must fulfill and the results of these checks are given as feedback to the engineer.

This simulator has been used in research and in courses to let students develop and test different adaptation techniques (e.g., hard-coded event-condition-action, graph transformation, or event-driven rules) for the analyze and plan steps. Though the simulator helped in finding faults in the adaptation logic, the randomness included in the simulator and its basic logging facilities impeded the automated reproducibility of traces and therefore, the retesting of “interesting” edge cases.

C. Online Testing and Validation

So far, we have not worked on testing the adaptation online. However, our experience with runtime models and employing MDE techniques at runtime for self-adaptation [6,20] gives us promising confidence to achieve the online testing. For instance, our EUREMA interpreter [20] that executes feedback loops already maintains the runtime models used within the loops and passes them along the loop’s adaptation activities. Thus, when passing models along the activities, the interpreter may defer the execution of the next activity. Before proceeding, the interpreter can either (1) hand over to an online testing activity that will compare the current RTM to one which is derived from a simulation model that runs in parallel or (2) log the RTM for later comparison in a simulation module. As discussed earlier, (1) has the advantage of immediate revelation of errors but needs computation resources on the system while (2) can benefit from more resources offline but needs persistence resources for the logs. In both cases, working only on changes of the RTM might reduce cost.

VIII. RELATED WORK

Testing of SASS has been addressed by others as well. This related work could usually be assigned to one of the following categories: 1) The adaptation is formally specified and verified with special constructs regarding the adaptation [21,22], 2) the SASS is tested/verified at runtime/online and the verification expressions are adapted to properties unique to adaptation [10]–[12], 3) tests are evolved at runtime in an attempt to test for requirement fulfillment even when the environment or the adaptable software changes [13,14], and 4) testing is carried out at design time addressing the special issues of adaptive systems [15]–[17]. 5) Combined assessment of quality assurance for self-adaptive systems from more than one direction has also been done [23].

The work presented in [23] already shows that a single quality assurance technique is not enough as an adequate

approach to achieve high-quality SASS. Testing and formal verification have long been known as complementing techniques for most kinds of systems. We assume that quality assurance for SASS can benefit in the same way from the combination of approaches like the one presented by us and approaches of category 1. Likewise, we see early testing as a complementary technique to online and adaptive online testing (cf. categories 2 and 3). To our understanding, this specifically holds for SASS where unknown circumstances may arise at runtime and need to be adequately taken care of. Nevertheless, testing still needs to be done before a system is to be deployed to ensure at least an initial and basic quality of the SASS.

Approaches of category 4 also address testing of SASS at design time. We differ from these approaches by not being dependent on a complete system. Using RTM as the test interface allows us to test already when there are only fragments of the system available, which is in the earlier development stages. Also our approach allows to test in a bottom-up manner, starting from the smallest testable units of a SASS and proceeding to the entire system.

IX. CONCLUSION

In this paper we presented a systematic testing scheme for SASS. It encompasses a staged testing process inspired by the engineering of embedded software. Exploiting architectural runtime models with their various states allows us to address the different stages of one-way, in-the-loop, and online testing. Supporting early development stages with tests, we may find errors early. Furthermore, looking at the individual MAPE-K activities and their different integrations, we should be able to locate faults more easily. In this context, our initial evaluation gives us preliminary confidence about the scheme's effectivity.

There are several directions to evolve the presented testing scheme in future. As of now we employ an ad hoc simulator for *RTMS*. We could instead make use of a formal model to automatically derive test cases by using coverage criteria, which includes the generation of test inputs and oracles (runtime models and properties) taking the uncertainty of SASS and its environment into account. Useful formalisms range from simple finite state machines to timed, hybrid or even probabilistic automata. Such a formal approach will further ease a thorough evaluation of the testing scheme. Another direction would be to address the neglected case that the adaptable software and environment change while the MAPE loop is running. We will study special test setups for this case.

REFERENCES

- [1] M. Salehie and L. Tahvildari, "Self-adaptive software: Landscape and research challenges," *ACM Trans. Auton. Adapt. Syst.*, vol. 4, no. 2, pp. 14:1–14:42, 2009.
- [2] R. de Lemos, H. Giese, H. Müller, M. Shaw, J. Andersson, M. Litoiu, B. Schmerl, G. Tamura, N. M. Villegas, T. Vogel, D. Weyns, L. Baresi, B. Becker, N. Bencomo, Y. Brun, B. Cukic, R. Desmarais, S. Dustdar, G. Engels, K. Geihs, K. Goeschka, A. Gorla, V. Grassi, P. Inverardi, G. Karsai, J. Kramer, A. Lopes, J. Magee, S. Malek, S. Mankovskii, R. Mirandola, J. Mylopoulos, O. Nierstrasz, M. Pezzè, C. Prehofer, W. Schäfer, R. Schlichting, D. B. Smith, J. P. Sousa, L. Tahvildari, K. Wong, and J. Wuttke, "Software Engineering for Self-Adaptive Systems: A second Research Roadmap," in *SEfSAS II*, ser. LNCS. Springer, 2013, vol. 7475, pp. 1–32.
- [3] R. Calinescu, "Emerging techniques for the engineering of self-adaptive high-integrity software," in *Assurances for Self-Adaptive Systems*, ser. LNCS. Springer, 2013, vol. 7740, pp. 297–310.
- [4] S. Nair, J. L. de la Vara, M. Sabetzadeh, and L. Briand, "An extended systematic literature review on provision of evidence for safety certification," *Information and Software Technology*, vol. 56, no. 7, pp. 689–717, 2014.
- [5] B. Broekman and E. Notenboom, *Testing embedded software*. Pearson Education, 2003.
- [6] T. Vogel, S. Neumann, S. Hildebrandt, H. Giese, and B. Becker, "Model-Driven Architectural Monitoring and Adaptation for Autonomic Systems," in *Proc. of the 6th Intl. Conference on Autonomic Computing and Communications (ICAC)*. ACM, 2009, pp. 67–68.
- [7] D. Garlan, S.-W. Cheng, A.-C. Huang, B. Schmerl, and P. Steenkiste, "Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure," *Computer*, vol. 37, no. 10, pp. 46–54, 2004.
- [8] J. Kramer and J. Magee, "Self-managed systems: An architectural challenge," in *Future of Software Engineering (FOSE)*. IEEE, 2007, pp. 259–268.
- [9] P. McKinley, S. M. Sadjadi, E. P. Kasten, and B. H. Cheng, "Composing Adaptive Software," *Computer*, vol. 37, no. 7, pp. 56–64, 2004.
- [10] H. J. Goldsby, B. H. Cheng, and J. Zhang, "Amoeba-rt: Run-time verification of adaptive software," in *Models in Software Engineering*. Springer, 2008, pp. 212–224.
- [11] Y. Zhao, S. Oberthür, M. Kardos, and F.-J. Rammig, "Model-based runtime verification framework for self-optimizing systems," *Electronic Notes in Theoretical Computer Science*, vol. 144, no. 4, pp. 125–145, 2006.
- [12] B. Eberhardinger, H. Seebach, A. Knapp, and W. Reif, "Towards testing self-organizing, adaptive systems," in *Testing Software and Systems*. Springer, 2014, pp. 180–185.
- [13] E. M. Fredericks and B. H. Cheng, "Automated generation of adaptive test plans for self-adaptive systems," in *Appear in Proceedings of 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, ser. SEAMS, vol. 15, 2015.
- [14] E. M. Fredericks, B. DeVries, and B. H. Cheng, "Towards run-time adaptation of test cases for self-adaptive systems in the face of uncertainty," in *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. ACM, 2014, pp. 17–26.
- [15] G. Püschel, C. Piechnick, S. Götz, C. Seidl, S. Richly, T. Schlegel, and U. Aßmann, "A combined simulation and test case generation strategy for self-adaptive systems," *Journal On Advances in Software*, vol. 7, no. 3&4, pp. 686–696, 2014.
- [16] Z. Wang, S. Elbaum, and D. S. Rosenblum, "Automated generation of context-aware tests," in *Software Engineering, 2007. ICSE 2007. 29th International Conference on*. IEEE, 2007, pp. 406–415.
- [17] J. Cámara, R. de Lemos, N. Laranjeiro, R. Ventura, and M. Vieira, "Testing the robustness of controllers for self-adaptive systems," *Journal of the Brazilian Computer Society*, vol. 20, no. 1, pp. 1–14, 2014.
- [18] G. Blair, N. Bencomo, and R. B. France, "Models@run.time," *Computer*, vol. 42, no. 10, pp. 22–27, 2009.
- [19] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [20] T. Vogel and H. Giese, "Model-Driven Engineering of Self-Adaptive Software with EUREMA," *ACM Trans. Auton. Adapt. Syst.*, vol. 8, no. 4, pp. 18:1–18:33, 2014.
- [21] M. Sama, D. S. Rosenblum, Z. Wang, and S. Elbaum, "Model-based fault detection in context-aware applications," in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. ACM, 2008, pp. 261–271.
- [22] M. U. Iftikhar and D. Weyns, "Formal verification of self-adaptive behaviors in decentralized systems with uppaal," Linnaeus University Växjö, Tech. Rep., 2012.
- [23] D. Weyns, "Towards an integrated approach for validating qualities of self-adaptive systems," in *Proceedings of the 2012 Workshop on Dynamic Analysis*. ACM, 2012, pp. 24–29.